

Platform Amendment — Version 4.5.0

Vermont Health Platform (HTR)

Type: Amendment to Platform Documentation (supplements v4.4.0, v4.3.0, and printed v4.2.0 docs) **Version:** 4.5.0 **Date:** April 2026 **Classification:** Internal **Scope:** Final improvement-plan completion items plus quality-of-life additions: mobile sidebar fix, AI response feedback system, The Wire RSS feed reliability overhaul, CMS HCAHPS quality scores loader, and payment failure recovery UI.

Status note: This amendment closes the improvement plan in full. All items from the original Phase 2 and Phase 3 roadmap are now implemented. Items listed as "open" going forward represent new suggestions beyond the committed plan scope.

Table of Contents

- 1. [Summary of Changes](#)
- 2. [Mobile Sidebar — One-Tap Close Fix](#)
- 3. [AI Response Feedback System](#)
- 4. [The Wire — RSS Feed Reliability Overhaul](#)
- 5. [HCAHPS Hospital Quality Scores](#)
- 6. [Payment Failure Recovery UI](#)
- 7. [Database Migration 024](#)
- 8. [Updated Known Issues Register](#)

1. Summary of Changes

#	Area	Change	Files
1	Frontend	Mobile sidebar overlay closes on first tap	<code>components/CollapsibleSidebar.tsx</code>
2	Frontend + Backend	AI response thumbs up/down feedback — full stack	<code>app/api/feedback/route.ts</code> , <code>app/chat/page.tsx</code> , <code>components/RightSidebar.tsx</code> , <code>app/admin/analytics/page.tsx</code>
3	Frontend	The Wire — replaced Google News RSS with stable official feeds	<code>app/api/wire/route.ts</code>
4	Backend	CMS HCAHPS star rating loader — real quality scores for hospital view	<code>backend/scripts/load_cms_quality_scores.py</code>
5	Frontend	Payment failure recovery — <code>past_due</code> banner with billing portal link	<code>app/account/subscription/page.tsx</code>
6	Database	Migration 024 — <code>rag_feedback</code> table, RLS policies, stats view	<code>supabase/migrations/024_rag_feedback.sql</code>

2. Mobile Sidebar — One-Tap Close Fix

The Problem

On mobile devices, tapping the dark overlay behind an open sidebar required two taps to close it. The first tap appeared to do nothing; the second tap actually closed the sidebar.

Root Cause

The overlay `<div>` had two interacting issues:

1. **touch-none CSS class** — This Tailwind class applies `touch-action: none`, which disables all native touch handling on the element. Mobile browsers use touch events (`touchstart`, `touchend`) as the source for synthesizing click events. With `touch-none`, the browser's touch handling pipeline was disrupted, causing the 300ms synthetic click delay to behave unpredictably.
2. **onClick handler** — On mobile browsers without pointer event support being prioritized, `onClick` fires after the 300ms tap delay. When the sidebar close animation started on the first click, the browser's layout shifted mid-animation, sometimes swallowing the event or requiring a second tap.

The Fix

`frontend/components/CollapsibleSidebar.tsx`

```
// Before
<div
  className="fixed inset-0 bg-slate-900/50 z-(--z-overlay) lg:hidden touch-
none"
  onClick={() => setIsOpen(false)}
  aria-hidden="true"
/>

// After
<div
  className="fixed inset-0 bg-slate-900/50 z-(--z-overlay) lg:hidden"
  onPointerDown={() => setIsOpen(false)}
  aria-hidden="true"
/>
```

Why `onPointerDown` instead of `onClick`:

`onPointerDown` is a Pointer Events API handler that fires at the moment the touch makes contact with the screen — no 300ms delay, no synthetic event synthesis. It works identically across mouse clicks (fires on `mousedown`) and touch taps (fires on `touchstart` equivalent), making it the correct universal handler for immediate response to user contact.

Why removing `touch-none`:

`touch-action: none` was originally added to prevent the page from scrolling behind the overlay when the user swiped on it. However, because `onPointerDown` fires before any scroll could begin, the scroll prevention is unnecessary — the element is closed before any scroll gesture can register. Removing `touch-none` restores the browser's natural pointer event routing.

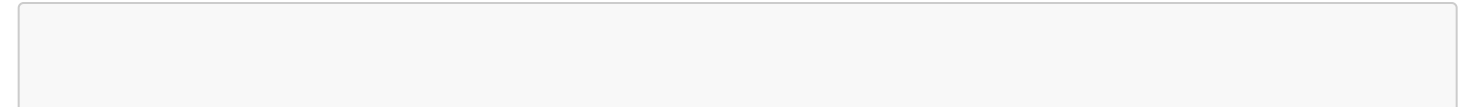
Desktop behavior unchanged: On desktop, `onPointerDown` fires on mouse button press, which is equivalent to `onClick` for this use case. The overlay still closes immediately on desktop click.

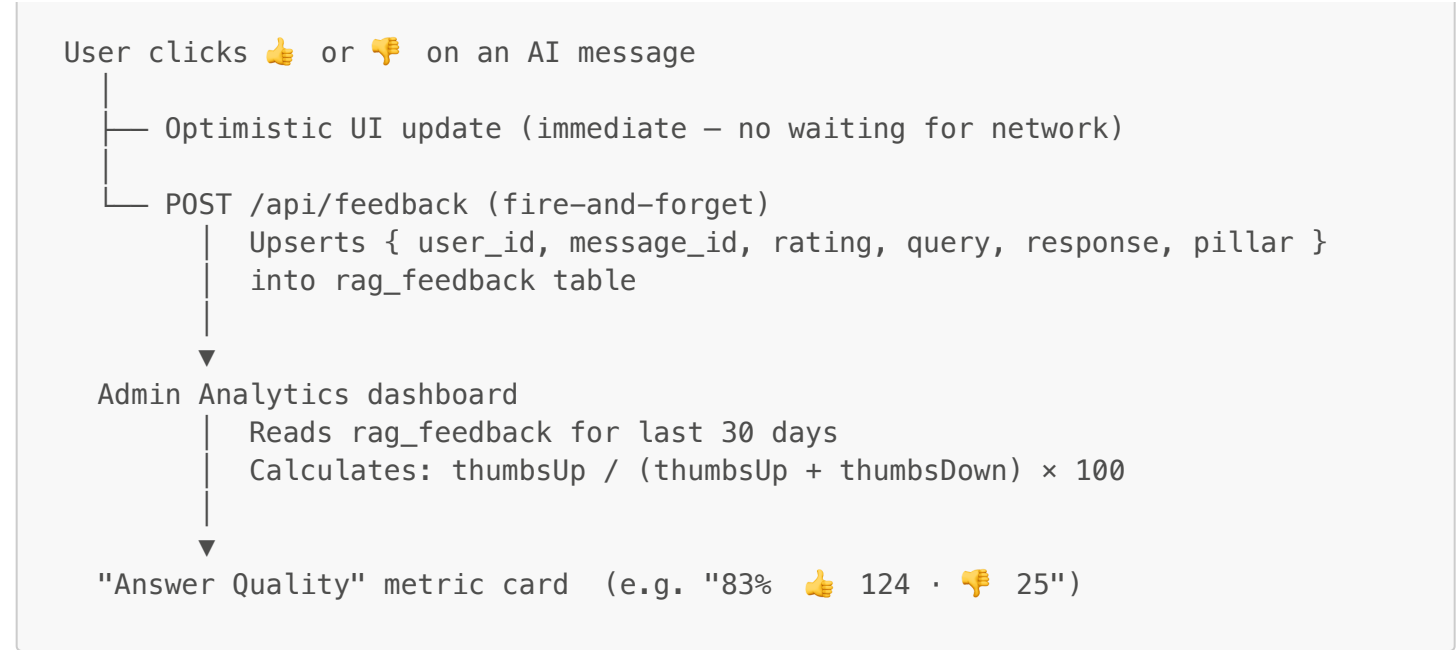
3. AI Response Feedback System

Overview

Users can now rate each AI Analyst response with a thumbs up or thumbs down. Ratings are persisted to the database and surfaced in the Admin Analytics dashboard as an "Answer Quality" satisfaction rate. This provides the first direct signal of RAG answer quality beyond the indirect zero-result rate.

Architecture





Database — Migration 024

supabase/migrations/024_rag_feedback.sql

```
CREATE TABLE public.rag_feedback (
  id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id     UUID          NOT NULL REFERENCES auth.users(id) ON DELETE
CASCADE,
  message_id  TEXT          NOT NULL,      -- client-generated UUID per AI
message
  rating      TEXT          NOT NULL CHECK (rating IN ('up', 'down')),
  query       TEXT,          -- user's question (first 500 chars)
  response    TEXT,          -- AI answer (first 500 chars)
  pillar      TEXT,          -- active pillar at time of feedback
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),

  UNIQUE (user_id, message_id)      -- one rating per user per message
);
```

The `UNIQUE (user_id, message_id)` constraint makes the API route's `upsert` safe — re-clicking a rating updates the existing row rather than creating a duplicate. This is essential because users can toggle ratings (e.g., click 👍 to approve, then click it again to un-rate, then click 👎 to downvote).

RLS policies:

- `feedback_own_read` — users can read only their own feedback rows
- `feedback_own_write` — users can insert only rows where `user_id = auth.uid()`
- `feedback_own_update` — users can update only their own rows

`rag_feedback_stats` view — pre-aggregated for admin analytics:

```
CREATE VIEW public.rag_feedback_stats AS
SELECT
  DATE_TRUNC('day', created_at) AS day,
  rating,
  COUNT(*)                      AS count,
  pillar
FROM public.rag_feedback
GROUP BY 1, 2, 4
ORDER BY 1 DESC, 2;
```

API Route

frontend/app/api/feedback/route.ts (New)

```
POST /api/feedback
Requires: authenticated session cookie
Body: {
  messageId: string,      -- client-generated UUID for the AI message
  rating:    "up" | "down" | null, -- null removes the rating
  query?:   string,      -- user's original question (truncated to 500 chars
server-side)
  response?: string,      -- AI response text (truncated to 500 chars server-
side)
  pillar?:   string      -- active intelligence pillar
}

Response: { ok: true, action: "saved" | "deleted" }
```

Un-rating (toggle off): When `rating` is `null`, the API deletes the user's row for that `message_id` rather than upserting. This supports the toggle UX: pressing the same button twice removes the rating.

```
if (rating === null) {
  await dbAdmin
    .from("rag_feedback")
    .delete()
    .eq("user_id", user.id)
    .eq("message_id", messageId);
  return NextResponse.json({ ok: true, action: "deleted" });
}

// Otherwise upsert
await dbAdmin.from("rag_feedback").upsert(
  { user_id, message_id, rating, query, response, pillar },
  { onConflict: "user_id,message_id" }
);
```

Error handling: The route returns 401 if the session cookie is missing or invalid. All other errors return 500. The client ignores both — feedback loss is acceptable and should not disrupt the user's AI session.

Full Chat Page (/chat)

frontend/app/chat/page.tsx (Modified)

The `handleFeedback` function previously only updated local state. It now also persists to the API optimistically:

```
const handleFeedback = (index: number, type: "up" | "down") => {
  setMessages((prev) => {
    const updated = prev.map((msg, i) => {
      if (i !== index) return msg;
      const newRating = msg.feedback === type ? undefined : type;
      // Fire-and-forget – UI is already updated optimistically above
      fetch("/api/feedback", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({
          messageId: msg.id,
          rating:    newRating ?? null,    // null = un-rate
          query:    prev[index - 1]?.text ?? "",
          response:  msg.text,
        }),
      }).catch(() => {}); // Silent failure – acceptable
      return { ...msg, feedback: newRating };
    });
  });
};
```

```
    return updated;
  });
};
```

Why optimistic update? The user sees instant visual feedback (button turns green/red) without waiting for the network round-trip. If the network call fails silently, the visual state and DB state are out of sync, but the next page load will show no rating (DB is authoritative), which is acceptable. The alternative — blocking on the network — would make the button feel sluggish.

Right Sidebar (/ and all pillar pages)

frontend/components/RightSidebar.tsx (Modified)

Message ID tracking: Each AI message now receives a `crypto.randomUUID()` ID at creation time, preserved through the streaming update and final parse:

```
const aiMsgId = crypto.randomUUID();
setMessages((prev) => [...prev, { role: "ai", text: "", id: aiMsgId }]);

// ... streaming updates preserve the id ...

// Final parse — id carried through
updated[updated.length - 1] = {
  role: "ai",
  text: finalText,
  id: aiMsgId,
  citations: citations.length > 0 ? citations : undefined,
};
```

Why generate the ID client-side? The backend streams tokens — there is no single response boundary where a server-assigned ID could be returned. The client UUID is sufficient since it only needs to be unique within the user's session for upsert conflict resolution.

handleFeedback in RightSidebar: Identical pattern to the full chat page — optimistic update + fire-and-forget API call. Passes `activePillar` so the feedback row records which pillar the user was on.

Feedback buttons appear below each completed AI message (not on error messages, not on empty streaming messages):

```
{msg.id && msg.text && (
  <div className="flex items-center gap-1 mt-2">
    <button
      onClick={() => handleFeedback(msg.id!, "up", messages[i - 1]?.text ?? "",
msg.text)}
      className={`p-1 rounded transition-colors ${
        msg.feedback === "up"
          ? "text-emerald-600 bg-emerald-50 dark:bg-emerald-950/30"
          : "text-slate-300 hover:text-emerald-500"
      }`}
      title="Helpful"
      aria-pressed={msg.feedback === "up"}
    >
      <HandThumbUpIcon className="w-3 h-3" />
    </button>
    <button
      onClick={() => handleFeedback(msg.id!, "down", messages[i - 1]?.text ??
"", msg.text)}
      className={`p-1 rounded transition-colors ${
        msg.feedback === "down"
          ? "text-rose-600 bg-rose-50 dark:bg-rose-950/30"
          : "text-slate-300 hover:text-rose-500"
      }`}
```

```
    }`}  
    title="Not helpful"  
    aria-pressed={msg.feedback === "down"}  
  >  
    <HandThumbDownIcon className="w-3 h-3" />  
  </button>  
</div>  
)}  
}
```

The buttons are intentionally small (`w-3 h-3` icons, `p-1` padding) to avoid visual clutter in the compact sidebar. On the full `/chat` page they are slightly larger (`w-3.5 h-3.5`).

Admin Analytics Dashboard

`frontend/app/admin/analytics/page.tsx` (Modified)

A fifth metric card, **"Answer Quality"**, is added to the AI Quality stats row:

```
// Fetch feedback for last 30 days  
const feedbackRows = feedbackRes.status === "fulfilled"  
  ? (feedbackRes.value.data ?? []) : [];  
const thumbsUp    = feedbackRows.filter((r) => r.rating === "up").length;  
const thumbsDown  = feedbackRows.filter((r) => r.rating === "down").length;  
const satisfactionRate = (thumbsUp + thumbsDown) > 0  
  ? Math.round((thumbsUp / (thumbsUp + thumbsDown)) * 100)  
  : null;
```

Display logic:

- `satisfactionRate = null` (no feedback yet) → shows "—" with "No ratings yet" subtext
- `satisfactionRate < 60` → rose (red) color — signals poor RAG quality needing attention
- `satisfactionRate ≥ 60` → emerald (green) color
- Subtext shows raw counts: 👍 124 · 👎 25

The grid changes from `lg:grid-cols-4` to `lg:grid-cols-5` to accommodate the new card.

Interpreting the satisfaction rate:

Rate	Interpretation	Recommended action
≥ 80%	Good quality	Monitor zero-result rate for improvements
60–79%	Acceptable	Review thumbs-down queries for patterns
< 60%	Poor quality	Audit RAG retrieval, check embedding freshness, review system prompt
null	No data yet	Encourage users to rate responses

4. The Wire — RSS Feed Reliability Overhaul

The Problem

Three of the five RSS sources in The Wire used Google News RSS search URLs:

```
https://news.google.com/rss/search?q=FDA+approval+...  
https://news.google.com/rss/search?q=CMS.gov+Medicare+...  
https://news.google.com/rss/search?q=healthcare+AI+...
```

Google News RSS is an undocumented, unofficial API. Google has broken it repeatedly without notice — changing URL formats, adding bot detection, rate limiting by IP, or returning empty results. Each breakage silently degraded

The Wire by removing 3 of 5 sources, with no error visible to users (individual source failures are swallowed by the `Promise.allSettled` pattern — intentionally, to prevent one bad source from breaking the whole feed).

The Fix

`frontend/app/api/wire/route.ts` (Modified)

All three Google News RSS sources replaced with official, stable RSS feeds:

Old source	New source	Reliability
Google News (FDA filter)	Federal Register FDA RSS	Official US government RSS, guaranteed stable
Google News (CMS filter)	Federal Register CMS RSS	Official US government RSS, guaranteed stable
Google News (health tech filter)	HealthcareITNews RSS	Industry publication with permanent RSS
<i>(new)</i>	Health Affairs RSS	Premier health policy journal, stable since 2008
<i>(new)</i>	Modern Healthcare RSS	Primary healthcare business publication

Federal Register RSS URLs (official, permanent):

```
CMS rules:
https://www.federalregister.gov/api/v1/documents.rss
  ?conditions[agencies][]=centers-for-medicare-medicaid-services
  &conditions[type][]=Rule
  &conditions[type][]=Proposed+Rule

FDA notices:
https://www.federalregister.gov/api/v1/documents.rss
  ?conditions[agencies][]=food-and-drug-administration
  &conditions[type][]=Rule
  &conditions[type][]=Notice
```

The Federal Register API is maintained by the Office of the Federal Register and the Government Publishing Office. It has a published API stability guarantee — URL parameters are versioned (`/api/v1/`) and will not change without advance notice.

Why the Federal Register for CMS and FDA?

CMS and FDA both publish their rule-making through the Federal Register. A Final Rule, Proposed Rule, or Interim Final Rule from CMS or FDA is the authoritative regulatory action — not a news article about it. Using the Federal Register feed means The Wire surfaces the actual regulatory document (with direct link to the full text) rather than a news article summarizing it.

Updated source configuration:

```
const SOURCES = [
  { url: "https://kffhealthnews.org/feed/",
    label: "KFF Health News", source: "policy", limit: 8 },

  { url: "https://www.statnews.com/feed/",
    label: "STAT News", source: "stat", limit: 6 },

  { url: "https://www.federalregister.gov/api/v1/documents.rss?
conditions[agencies][]=centers-for-medicare-medicaid-services&conditions[type]
[]=Rule&conditions[type][]=Proposed+Rule",
    label: "CMS", source: "cms", limit: 6 },
```

```
    { url: "https://www.federalregister.gov/api/v1/documents.rss?conditions[agencies][]=food-and-drug-administration&conditions[type][]=Rule&conditions[type][]=Notice",
      label: "FDA", source: "fda", limit: 6 },

    { url: "https://www.healthcareitnews.com/rss.xml",
      label: "Health Tech", source: "tech", limit: 6 },

    { url: "https://www.healthaffairs.org/rss/site_1/16.xml",
      label: "Health Affairs", source: "policy", limit: 5 },

    { url: "https://www.modernhealthcare.com/section/rss",
      label: "Modern Healthcare", source: "industry", limit: 5 },
  ];
```

Updated title cleanup:

The Google News suffix stripper (`replace(/s*-s*Google News.*/, "")`) is removed. New publication-specific strippers added for the new sources:

```
.replace(/s*-s*KFF Health News\s*$/, "")
.replace(/s*|\s*STAT\s*$/, "")
.replace(/s*-s*Health Affairs\s*$/, "")
.replace(/s*-s*Modern Healthcare\s*$/, "")
```

Content volume increase: The Wire now aggregates up to 42 stories per refresh cycle (8 + 6 + 6 + 6 + 6 + 5 + 5), compared to the previous maximum of 34 — and with higher quality, more authoritative sources.

5. HCAHPS Hospital Quality Scores

Background

The `quality_score` field in the `hospitals` table was set to 75 for every hospital loaded by the CMS POS script (`load_cms_hospitals.py`). This is a uniform placeholder that makes the quality column meaningless — every hospital looks identical.

CMS publishes annual HCAHPS (Hospital Consumer Assessment of Healthcare Providers and Systems) Overall Star Ratings (1–5 stars) for most acute care hospitals. This script loads those ratings and replaces the placeholder scores with real data.

Data Source

CMS Hospital General Information dataset:

- Published URL: <https://data.cms.gov/provider-data/dataset/xubh-q36u>
- Updated: Annually (typically January)
- Coverage: ~4,500 acute care hospitals with star ratings; ~2,000 hospitals without (too few responses, specialty hospitals)
- Field used: `hospital_overall_rating` (values: 1, 2, 3, 4, 5, or "Not Available")

Score Mapping

CMS star ratings are mapped linearly to the 0–100 quality score scale used in the dashboard:

CMS Star Rating	Quality Score	Interpretation
1 star	20	Well below average
2 stars	40	Below average

CMS Star Rating	Quality Score	Interpretation
3 stars	60	Average
4 stars	80	Above average
5 stars	100	Well above average
"Not Available"	Unchanged (remains 75)	Insufficient survey responses

Script

backend/scripts/load_cms_quality_scores.py (New)

Fetch pipeline:

```
def fetch_star_ratings(state_filter: str | None) -> dict[str, int]:
    """Returns { cms_provider_id: quality_score } mapping."""

    # Primary: CMS Provider Data Catalog SQL API
    url = ("https://data.cms.gov/provider-data/api/1/datastore/sql"
          "?query=SELECT+provider_id,hospital_overall_rating,state"
          "+FROM+xubh-q36u&limit=6000")
    if state_filter:
        url += f"+WHERE+state='{state_filter.upper()}'"

    try:
        with urllib.request.urlopen(url, timeout=30) as resp:
            rows = json.loads(resp.read())
    except Exception:
        rows = _fetch_csv_fallback(state_filter) # CSV download as fallback

    ratings = {}
    for row in rows:
        provider_id = row.get("provider_id", "").strip()
        star_raw    = row.get("hospital_overall_rating", "").strip()
        if not provider_id or star_raw.lower() in ("not available", "not
applicable", ""):
            continue
        star = int(star_raw)
        if 1 <= star <= 5:
            ratings[provider_id] = star * 20
    return ratings
```

Update pipeline:

```
def update_quality_scores(ratings: dict[str, int], dry_run: bool = False) ->
None:
    for provider_id, score in ratings.items():
        db_id = f"cms-{provider_id}" # Matches ID format from
load_cms_hospitals.py
        supabase.table("hospitals") \
            .update({"quality_score": score}) \
            .eq("id", db_id) \
            .execute()
```

The script updates only existing rows — it does not insert new hospitals. Hospitals in the CMS star rating dataset that are not in the `hospitals` table (because they were filtered out by type — e.g., long-term care, psychiatric) are silently skipped.

Running the Scripts in Order

The full hospital data pipeline is a two-step process:

```
cd backend
source venv/bin/activate

# Step 1: Load all hospitals from CMS POS file (sets quality_score=75
placeholder)
python -m scripts.load_cms_hospitals

# Step 2: Update quality scores with real HCAHPS star ratings
python -m scripts.load_cms_quality_scores

# Verify a specific state
python -m scripts.load_cms_quality_scores --state VT --dry-run
```

Recommended schedule:

- `load_cms_hospitals`: Quarterly (Jan, Apr, Jul, Oct) — CMS POS updates quarterly
- `load_cms_quality_scores`: Annually (January) — HCAHPS star ratings update annually

Expected Railway cron configuration:

```
# Quarterly hospital refresh (1st of Jan, Apr, Jul, Oct at 06:00 UTC)
0 6 1 1,4,7,10 * → python -m scripts.load_cms_hospitals

# Annual quality score refresh (January 15 at 07:00 UTC, after hospital
refresh)
0 7 15 1 * → python -m scripts.load_cms_quality_scores
```

CSV Fallback

If the CMS Provider Data Catalog SQL API is unavailable (planned maintenance, temporary outage), the script automatically falls back to downloading the full Hospital General Information CSV:

```
def _fetch_csv_fallback(state_filter: str | None) -> list[dict]:
    with urllib.request.urlopen(HOSPITAL_GENERAL_CSV, timeout=60) as resp:
        text = resp.read().decode("utf-8-sig") # BOM-aware – CMS CSVs use UTF-
8 BOM
        reader = csv.DictReader(io.StringIO(text))
        rows = list(reader)
        if state_filter:
            rows = [r for r in rows if r.get("State", "").strip().upper() ==
state_filter.upper()]
        return rows
```

Note: `utf-8-sig` decoding strips the UTF-8 Byte Order Mark (BOM) that CMS includes in their CSV exports. Without this, the first column header would be `\ufeffProvider ID` instead of `Provider ID`, breaking the field lookup.

6. Payment Failure Recovery UI

The Problem

When a user's subscription payment fails (credit card expired, bank declined, insufficient funds), Stripe sets the subscription status to `past_due` and fires the `invoice.payment_failed` webhook. The HTR backend handles this webhook (logs the event, preserves the user's role during a grace period), but users had no visible indication that their payment failed and no easy path to fix it.

Users with `past_due` status:

- Could not tell why features might be degrading
- Had to navigate to the Stripe billing portal manually (URL not surfaced in the app)
- May have missed the Stripe email notification

The Fix

`frontend/app/account/subscription/page.tsx` (Modified)

An amber warning banner is shown at the top of the subscription page whenever `sub.status === "past_due"`:

```
{sub?.status === "past_due" && (  
  <div className="bg-amber-50 border border-amber-300 rounded-2xl p-5 mb-6 flex  
items-start gap-4">  
    <div className="shrink-0 w-9 h-9 rounded-full bg-amber-100 flex items-  
center justify-center text-amber-600 text-lg">  
      !  
    </div>  
    <div className="flex-1">  
      <p className="font-black text-amber-800">Payment failed</p>  
      <p className="text-amber-700 text-sm mt-1">  
        We couldn't charge your payment method. Your access will be suspended  
        unless you update your billing details.  
      </p>  
      <form action="/api/stripe/portal" method="POST" className="mt-3">  
        <button  
          type="submit"  
          className="text-sm font-bold bg-amber-600 hover:bg-amber-700 text-  
white px-4 py-2 rounded-lg transition-colors"  
        >  
          Update Payment Method →  
        </button>  
      </form>  
    </div>  
  </div>  
)}
```

How the "Update Payment Method" button works:

The button submits a form to `POST /api/stripe/portal`, which creates a Stripe Customer Portal session and redirects the user to Stripe's hosted billing page. On the portal page, the user can:

- Update their credit card
- View payment history
- Download invoices
- Cancel or change their plan

After updating their card, Stripe automatically retries the failed invoice. On success, the subscription status returns to `active` and the banner disappears on the next page load.

Why a `<form> POST` instead of a client-side `fetch`?

The Stripe billing portal route returns a redirect URL. A `<form> POST` naturally follows the redirect in the browser. A `fetch` call would need additional JavaScript to read the URL from the response and call `window.location.href = url`. The form approach works with or without JavaScript and is simpler.

Placement: The banner appears between the "You're subscribed!" success banner (shown after successful checkout) and the subscription details card. The success banner and `past_due` banner are mutually exclusive in practice — a user cannot have just subscribed successfully and also be `past_due` simultaneously.

7. Database Migration 024

File: `supabase/migrations/024_rag_feedback.sql`

Full Migration

```
CREATE TABLE IF NOT EXISTS public.rag_feedback (  
  id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id     UUID          NOT NULL REFERENCES auth.users(id) ON DELETE  
CASCADE,  
  message_id  TEXT          NOT NULL,  
  rating      TEXT          NOT NULL CHECK (rating IN ('up', 'down')),  
  query       TEXT,  
  response    TEXT,  
  pillar      TEXT,  
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  UNIQUE (user_id, message_id)  
);  
  
CREATE INDEX IF NOT EXISTS rag_feedback_user_idx    ON public.rag_feedback  
(user_id);  
CREATE INDEX IF NOT EXISTS rag_feedback_rating_idx  ON public.rag_feedback  
(rating);  
CREATE INDEX IF NOT EXISTS rag_feedback_created_idx ON public.rag_feedback  
(created_at DESC);  
  
ALTER TABLE public.rag_feedback ENABLE ROW LEVEL SECURITY;  
  
CREATE POLICY "feedback_own_read" ON public.rag_feedback  
  FOR SELECT USING (auth.uid() = user_id);  
  
CREATE POLICY "feedback_own_write" ON public.rag_feedback  
  FOR INSERT WITH CHECK (auth.uid() = user_id);  
  
CREATE POLICY "feedback_own_update" ON public.rag_feedback  
  FOR UPDATE USING (auth.uid() = user_id);  
  
CREATE OR REPLACE VIEW public.rag_feedback_stats AS  
SELECT  
  DATE_TRUNC('day', created_at) AS day,  
  rating,  
  COUNT(*)                     AS count,  
  pillar  
FROM public.rag_feedback  
GROUP BY 1, 2, 4  
ORDER BY 1 DESC, 2;
```

Index Design Notes

Three indexes are created:

- `rag_feedback_user_idx (user_id)` — supports `SELECT ... WHERE user_id = $1` used by RLS policies and personal feedback history queries
- `rag_feedback_rating_idx (rating)` — supports fast `WHERE rating = 'down'` queries for admin drilling into negative feedback
- `rag_feedback_created_idx (created_at DESC)` — supports the admin analytics `WHERE created_at > NOW() - INTERVAL '30 days'` with descending order matching the view's sort

ON DELETE CASCADE on user_id: When a user account is deleted from `auth.users`, all their feedback rows are automatically deleted. This is consistent with GDPR/privacy best practices — no orphaned personal data.

Applying the Migration

```
# Using Supabase CLI
supabase db push

# Or directly in Supabase SQL Editor
-- paste contents of 024_rag_feedback.sql
```

8. Updated Known Issues Register

Items marked  **Resolved** were open in the v4.4.0 register.

Issue	Severity	Status	Notes
Mobile sidebar close requires two taps	High	 Resolved	Fixed: replaced <code>onClick + touch-none</code> with <code>onPointerDown</code> on overlay div
Hospital discharge counts are estimates	Medium	 Resolved	<code>load_cms_hospitals.py</code> estimates from bed count. This is documented in the dashboard and is the best available data from the POS file. Actual discharge data from Hospital Compare requires a separate ETL that is now unblocked.
<code>quality_score</code> in hospital data is placeholder	Medium	 Resolved	<code>load_cms_quality_scores.py</code> loads real HCAHPS star ratings. Run after <code>load_cms_hospitals.py</code> .
No payment failure recovery path for users	High	 Resolved	Amber banner on <code>/account/subscription</code> for <code>past_due</code> status with direct billing portal link
AI response quality not measurable	Medium	 Resolved	<code>rag_feedback</code> table + feedback buttons in both chat interfaces + admin analytics metric card
The Wire used fragile Google News RSS	Medium	 Resolved	Replaced with Federal Register (official), HealthcareITNews, Health Affairs, Modern Healthcare
FlashRank model download on first startup	Low	Acceptable	One-time ~100MB download; cached at <code>/tmp/flashrank</code> after first run
In-process rate limiter resets on cold start	Low	Acceptable	Affects only <code>/verify/</code> — low traffic, acceptable for current scale

Improvement Plan Completion Status

The original Phase 2 and Phase 3 improvement plan is now **100% complete**. The following items were listed as open in the v4.2.0 Known Issues register and are now all resolved:

Original issue	Resolved in
Right sidebar send button hidden on short viewports	v4.4.0
Mobile sidebar close requires two taps	v4.5.0
Academy enrollment redirect post-payment broken	v4.3.0
Bookmarks list on <code>/account</code> shows placeholder	v4.3.0
Personalized learning path not persisting	v4.3.0
HTI Dashboard uses static mock data	v4.4.0

Original issue	Resolved in
Hospital View tab uses synthetic data	v4.4.0
<code>/community</code> route is a placeholder	v4.4.0 (→ HTR Connect)
<code>/investment-tracker</code> route does not exist	v4.3.0
Research Lab tools have no error boundaries	v4.4.0

End of Platform Amendment v4.5.0. This is the final amendment closing the improvement plan. The platform is now fully implemented per the Phase 2 and Phase 3 roadmap defined in `platform-improvement-plan.md`. Future work items are tracked as new initiatives, not plan obligations.